Day 1 Task Workflow — Tuesday, Feb 25, 2026

> **Sprint:** 1 (Foundation) | **Phase:** Project Initialization
> **Owner:** Lead Architect | **Status:** 🟢 In Progress
> **Goal:** Standing monorepo with running backend API + frontend shell + Docker infrastructure

---

Today's Objective

Set up the complete project scaffolding so that by end of day:

- `uvicorn app.main:app --reload` serves API docs at `localhost:8000/docs`
- `npm run dev` serves the Next.js dark shell at `localhost:3000`
- `docker-compose up` spins up PostgreSQL + Redis
- All directory structures match the master doc specification

---

Task Breakdown

TASK 1 — Backend Project Structure ☑

Time Est: 30 min | **Priority:** 🔴 Critical

Create `backend/` with FastAPI project layout:

File	Purpose	Status
`backend/requirements.txt`	Pinned Python dependencies	☑ Done
`backend/Dockerfile`	Container build for backend service	☑ Done
`backend/app/__init__.py`	Package marker	☑ Done
`backend/app/config.py`	`pydantic-settings` BaseSettings (DB, Redis, JWT, LLM keys, capture)	☑ Done
`backend/app/database.py`	Async SQLAlchemy engine, session factory, `get_db` dependency	☑ Done
`backend/app/main.py`	FastAPI app factory, CORS, lifespan, router mount	☑ Done
`backend/app/api/v1/__init__.py`	Router aggregator (auth, system, flows, alerts)	☑ Done
`backend/app/api/v1/system.py`	`/health` and `/info` endpoints	☑ Done

File	Purpose	Status
`backend/app/api/v1/auth.py`	Auth stubs (register, login, refresh, me) with Pydantic schemas	☑ Done
`backend/app/api/v1/flows.py`	Flow endpoint stubs (list, stats, top-talkers, protocols)	☑ Done
`backend/app/api/v1/alerts.py`	Alert endpoint stubs (list, stats, detail, status update)	☑ Done

Verification: `cd backend && uvicorn app.main:app --reload --port 8000` → visit `localhost:8000/docs`

TASK 2 — Docker Infrastructure ☑

Time Est: 20 min | **Priority:** ⬤ Critical

File	Purpose	Status
`docker-compose.yml`	Production stack: PostgreSQL 16, Redis 7, Backend, ML Worker	☑ Done
`docker-compose.dev.yml`	Dev overrides: hot reload, volume mounts	☑ Done
`.env.example`	Environment variable template	☑ Done

Verification: `docker-compose up -d postgres redis` → PostgreSQL on `:5432`, Redis on `:6379`

TASK 3 — Frontend Initialization 🔨

Time Est: 45 min | **Priority:** ⬤ Critical

Initialize Next.js 16 with App Router, TypeScript strict, no Tailwind:

Step	Action	Status
3.1	Run `npx create-next-app@latest frontend` (TypeScript, App Router, no Tailwind, no src dir)	☐
3.2	Create `styles/globals.css` — full design system (CSS variables, fonts, animations)	☐
3.3	Create `app/layout.tsx` — root layout with dark theme, font imports	☐
3.4	Create `app/page.tsx` — redirect to `/war-room`	☐
3.5	Verify `npm run dev` → dark page at `localhost:3000`	☐

Design System CSS must include:

- Color tokens: `--bg-primary`, `--cyan`, `--critical`, `--safe`, etc.
- Typography: JetBrains Mono (data), Inter (UI)
- Glass panel mixin: backdrop blur + subtle border

- Scan-line animation keyframe
- Pulse animation keyframe for critical alerts

TASK 4 — Makefile 🔨

Time Est: 10 min | **Priority:** 🟢 Medium

Common dev commands:

```
dev-backend    # uvicorn with reload
dev-frontend   # npm run dev
docker-up      # docker-compose up -d postgres redis
docker-down    # docker-compose down
test           # pytest
lint           # ruff + eslint
```

TASK 5 — Verify Full Stack 🔨

Time Est: 15 min | **Priority:** 🔴 Critical

End-of-day integration check:

Check	Command	Expected	Status
PostgreSQL running	`docker-compose up -d postgres`	Port 5432 open	☐
Redis running	`docker-compose up -d redis`	Port 6379 open	☐
Backend API	`uvicorn app.main:app --reload`	Swagger at `:8000/docs`	☐
Health endpoint	`curl localhost:8000/api/v1/system/health`	JSON response	☐
Frontend shell	`npm run dev`	Dark page at `:3000`	☐

Files Created Today

```
threatmatrix-ai/
├── backend/
│   ├── app/
│   │   ├── __init__.py          ☑
│   │   ├── config.py            ☑
│   │   ├── database.py          ☑
│   │   ├── main.py              ☑
│   │   ├── api/v1/
│   │   │   ├── __init__.py      ☑
```

```
│   │   │   ├── system.py        ☑
│   │   │   ├── auth.py          ☑
│   │   │   ├── flows.py         ☑
│   │   │   └── alerts.py        ☑
│   │   ├── models/              (empty, Day 2)
│   │   ├── schemas/             (empty, Day 2)
│   │   └── services/            (empty, Day 3)
│   ├── capture/                 (empty, Week 2)
│   ├── ml/                      (empty, Week 3)
│   ├── requirements.txt         ☑
│   └── Dockerfile               ☑
├── frontend/                    🔨 Task 3
├── docker-compose.yml           ☑
├── docker-compose.dev.yml       ☑
├── .env.example                 ☑
└── Makefile                     🔨 Task 4
```

Decisions Made Today

Decision	Choice	Reason
Async SQLAlchemy	`asyncpg` + `create_async_engine`	Non-blocking DB ops for real-time pipeline
API stub pattern	Return empty structures with correct shapes	Frontend can start building against API contract immediately
Auth deferred	Stubs with 501 responses	Full JWT impl on Day 3, don't block Day 1 scaffolding

---

Blockers

None.

Tomorrow's Preview (Day 2)

- SQLAlchemy ORM models for all tables (`users`, `network_flows`, `alerts`, etc.)
- Alembic init + initial migration
- Database tables created and verified
- Pydantic request/response schemas

Task workflow for Day 1 — ThreatMatrix AI Sprint 1